SOFTWARE PRODUCT · DOCUMENTATION

FINDXNY OS

POS & operations platform

A multi-tenant POS and booking platform spanning web, Expo cashier terminals, Supabase backend infrastructure, offline workflows, payments, and physical POS hardware.

ROLE	PERIOD	PLATFORMS
Junior Full-Stack Engineer	**May 2026 – Jul 2026**	**Web · POS · Backend · Hardware**

CONTENTS

1. Overview
2. Scope and personal contribution
3. Product surfaces
4. System architecture
5. Technology stack
6. Capabilities
7. Engineering deep dives
8. Data model
9. Engineering decisions
10. System scale
11. Evidence, status and limitations

Prince John Gandollas · Full-Stack Engineer
pgandollas@gmail.com · LinkedIn · GitHub · pjdevph.vercel.app

Generated from the portfolio's content on 2026-10-12. Live version: pjdevph.vercel.app/projects/findxny-os

1. Overview

A multi-tenant POS and booking platform spanning web, Expo cashier terminals, Supabase backend infrastructure, offline workflows, payments, and physical POS hardware.

PROJECT	FINDXNY OS
TYPE	POS & operations platform
ROLE	Junior Full-Stack Engineer
PERIOD	May 2026 – Jul 2026
PLATFORMS	Web · POS · Backend · Hardware

2. Scope and personal contribution

> Counts such as "114 Edge Functions" describe the whole platform, not only Prince's own work. His own contributions are the areas of ownership listed in this section.

- Offline sync & idempotent replay
- Payment webhooks & dedup
- Tenant/role access (RLS)
- POS hardware integrations

What was built, in detail:

- Designed workspace-, branch-, and role-based access for owners, admins, managers, cashiers, and kitchen staff, enforced by PostgreSQL RLS and shared permission-group checks in every Edge Function.
- Built a SQLite-backed offline sync queue (orders, payments, shift actions) with idempotency keys, a capped retry count, and a stuck-order recovery view for unreliable connectivity.
- Authored native Expo modules for ESC/POS printing over USB/Bluetooth, IMIN built-in thermal printers, physical cash-drawer triggering, and a secondary customer-facing display.
- Implemented Xendit Invoice webhooks with a unique-constraint dedup table, request-hash idempotency on mutating endpoints, and server-side pricing so clients never compute totals.

3. Product surfaces

Customer Web

`Next.js`

Public storefront for a FINDXNY tenant — menu browsing, cart, staycation booking, and an order/booking tracker.

- (customer) route group in the Next.js app
- Menu, staycation booking, order tracking pages
- Tenant-branded deployment (mugthemug.ph)

Admin Web

`Next.js · role-gated`

A separate (admin) route group in the same Next.js app for catalog, staff, reports, and workspace settings.

- Product/catalog, inventory & costing management
- Staff, roles, and audit log views
- Sales reports and Z-report review

POS + Kitchen App

`Expo · React Native`

The terminal cashiers and kitchen staff use — order entry, kitchen tickets, shifts, and hardware I/O, in one app.

- Offline-first checkout via a local SQLite queue
- ESC/POS + IMIN thermal printing, physical cash drawer
- Shift management, kitchen ticket board, secondary display

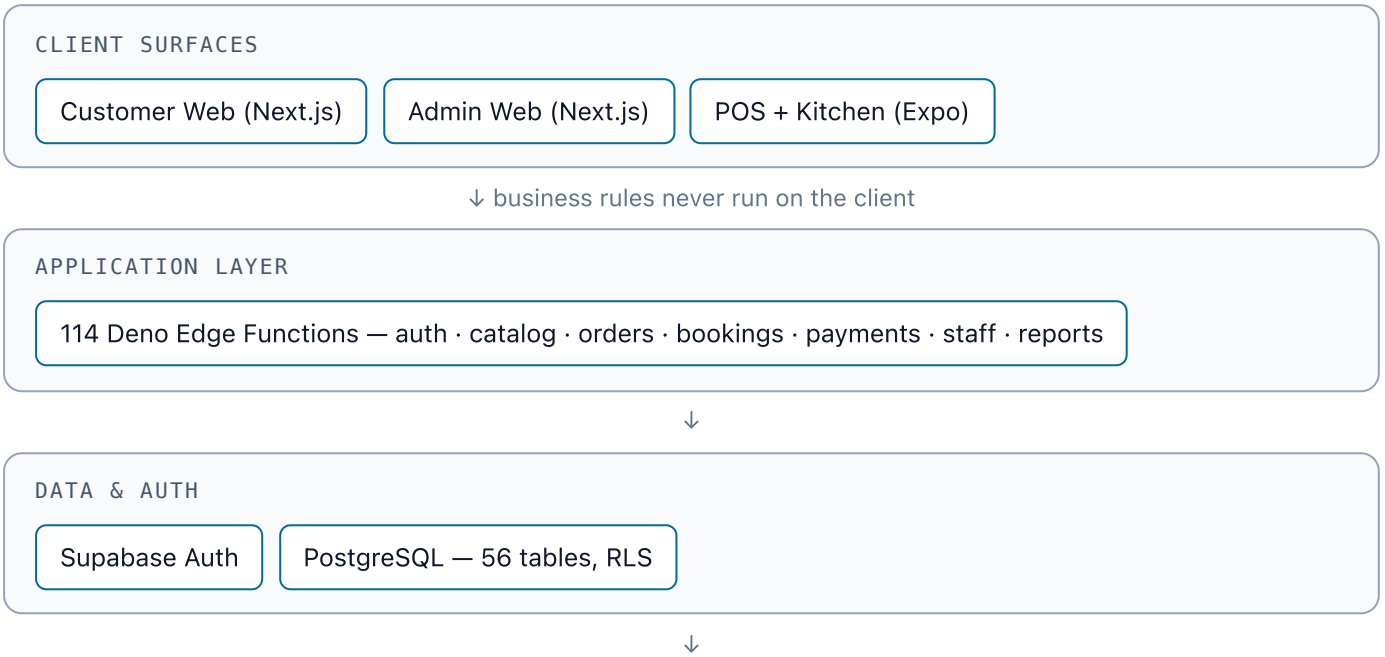
Backend Platform

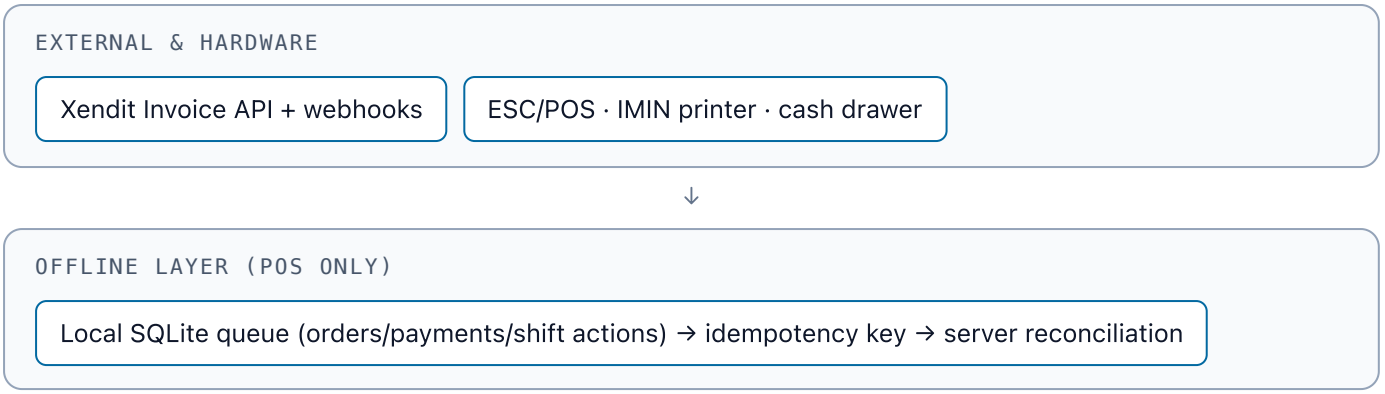
`Supabase · Deno`

The shared core every client calls into: auth, tenant-isolated Postgres data, pricing, and payment webhooks.

- 114 Deno Edge Functions across 10 domains
- PostgreSQL — 56 tables, 121 foreign keys, RLS
- Xendit webhook processing with dedup

4. System architecture





> There is no local Supabase/Docker stack: development, staging, and production are all real deployed Supabase projects (per the repository README).

5. Technology stack

Area	Technologies
Web	Next.js 14, React 18, Tailwind CSS 4
POS Client	Expo (React Native 0.83), expo-router, expo-sqlite
Backend	Deno Edge Functions, Supabase
Data & Auth	PostgreSQL, Row-Level Security, Supabase Auth
Payments	Xendit Invoice API, Webhooks
Testing & Delivery	Vitest, Playwright, EAS Build/Update
POS Hardware	ESC/POS (USB/Bluetooth), IMIN thermal SDK, Native cash drawer

6. Capabilities

Customer & Booking

- Public storefront, menu, and cart per tenant (e.g. mugthemug.ph)
- Staycation/resource booking with holds, check-in/out, and reschedule
- Order & booking tracker by order number + mobile number

POS & Kitchen

- Order entry, per-item kitchen routing, and a live kitchen ticket board
- Shift open/close, clock-in/out, and per-shift cash-drawer reconciliation
- Loyalty points, vouchers, and manager-approval overrides for voids/refunds

Platform & Access

- Multi-tenant workspace → branch model enforced by PostgreSQL RLS
- Owner / admin / manager / cashier / kitchen roles via named permission groups
- PIN-based staff login with rate-limited auth attempts (no plaintext PINs stored)

Payments & Operations

- Xendit Invoice checkout (GCash, Maya, card, QRPh) plus a separate pay-at-counter cash flow
- Idempotency-Key handling on mutating endpoints, with request-hash collision detection
- Z-reports, sales analytics, inventory/stock adjustments, and expense tracking

7. Engineering deep dives

Offline-first checkout: how an order survives a network failure

1. Cashier checks out; order and payment bodies are written to local SQLite tables first
2. If offline, the operation queues with a generated idempotency key
3. A background sync service replays the queue on reconnect — orders, payments, then shift actions
4. The server validates each idempotency key before applying it, so a resent order can't double-post
5. An order that fails 8 times surfaces to the cashier as "stuck" instead of retrying forever

Payments: how a payment is confirmed exactly once

1. payments-create-intent computes the total server-side and opens a Xendit Invoice (or routes to the cash-at-counter flow)
2. Xendit hosts checkout across GCash, Maya, card, and QRPh behind one invoice
3. Xendit calls payments-webhook on completion
4. The handler inserts into webhook_events first — a unique constraint on (invoice, status) lets exactly one delivery win and silently drops retries
5. payment_intents, then the order or booking, are updated — paid orders auto-spawn a kitchen ticket for kitchen-required items
6. A transaction + receipt row is recorded, and a booking receipt is emailed automatically when a guest email is on file

8. Data model

Selected tables, grouped by domain (the full schema has 56 tables):

Domain	Tables
WORKSPACE & ACCESS	workspaces, branches, workspace_members, profiles, pos_devices
ORDERS & KITCHEN	orders, order_items, order_charges, kitchen_tickets, receipts
PAYMENTS & TRANSACTIONS	payment_intents, payments, transactions, refunds

Domain	Tables
BOOKINGS & RESOURCES	bookings, bookable_resources, resource_blocks
SHIFTS & CASH MANAGEMENT	shifts, shift_events, cash_drawer_events, z_reports
CATALOG & INVENTORY	products, product_variants, inventory_items, stock_movements

9. Engineering decisions

Offline transaction durability

PROBLEM

Cashiers still need to transact when internet access drops.

DECISION

Persist pending operations in SQLite and replay them with idempotency keys when connectivity returns.

IMPACT

Transactions survive app restarts and network interruptions without double-charging or double-posting.

Tenant isolation

PROBLEM

Many businesses and branches share the same infrastructure.

DECISION

Enforce tenant boundaries in PostgreSQL RLS and edge middleware instead of trusting client-side filtering.

IMPACT

A compromised or buggy client cannot freely cross workspace boundaries.

10. System scale

- 114 Edge Functions
- 119 DB migrations
- 56 tables · 121 FKs
- Offline SQLite queue
- ESC/POS + cash drawer

> These figures describe the whole system. His own contributions are the areas of ownership listed in this section.

11. Evidence, status and limitations

- This document describes architecture and behaviour only. It contains no credentials, internal endpoints or customer data.

- No performance, revenue or user-count figures are published for this project.
- The platform's public tenant storefront is temporarily offline, so its link is not shown. Online food ordering and staycation booking had been paused.
- Screenshots and a walkthrough video are on the case-study page.

Sources:

- Case study: https://pjdevph.vercel.app/projects/findxny-os
- Redacted code showcase: https://github.com/pjDevph/findxny-os-showcase